# Reinforcement Learning as an Alternative to Simulated Annealing for MaxSAT

**Andrej Ohrablo**
aohrablo@uwaterloo.ca

## Abstract

This paper investigates the applicability of reinforcement learning-specifically Deep Q-Networks (DQN)-to the NP-hard Maximum Satisfiability (MaxSAT) problem, and compares its performance to a simulated annealing baseline of similar computational complexity. We evaluate both approaches on problem sets of increasing difficulty to assess scalability and solution quality. Our results show that the DQN-based method is capable of learning meaningful search behaviors and ultimately achieves performance comparable to that of a manually tuned simulated annealing algorithm.

## 1 Introduction

The **Maximum Satisfiability (MaxSAT)** problem extends classical SAT by seeking an assignment that satisfies the maximum number of clauses. This formulation appears in practical settings such as preference reasoning, hardware verification, and constrained optimization problems where not all constraints can be satisfied simultaneously. As with SAT, the search space grows exponentially with the number of variables, and MaxSAT remains **NP-hard**, making exact solvers impractical for larger instances. Consequently, many approaches rely on local-search strategies, including simulated annealing, which can navigate the landscape reasonably well but still depends heavily on manually chosen schedules and handcrafted scoring rules.

An alternative direction is to let the search procedure *learn* how to move through MaxSAT landscapes instead of relying on fixed heuristics. Reinforcement learning (RL) has been explored in several combinatorial settings for this reason: the agent repeatedly interacts with instances, observes which moves are beneficial, and gradually forms a policy that reflects the structure of the underlying problem. In contrast to generic local-search heuristics, an RL agent can adapt its behavior as it encounters clauses of differing difficulty or problem-specific patterns that are hard to encode manually.

Among RL methods, **model-free approaches such as Deep Q-Networks (DQN)** fit well with the nature of MaxSAT. The action space-flipping a single variable-is discrete and relatively small, which aligns with Q-learning's strengths. Model-free RL are also less computationally expensive and often can be run locally, as opposed to the model-based approach that requires simulating or approximating the effects of each variable flip. A DQN sidesteps this by learning value estimates directly from the observed improvement or deterioration of the assignment. Over time, the network can pick up patterns such as which variables tend to influence the most unsatisfied clauses or when it is worthwhile to escape a locally satisfying configuration. This gives DQN enough flexibility to compete with manually tuned simulated annealing while keeping the overall method relatively simple.

## 2 Related Work

Reinforcement learning has been explored as a tool to enhance heuristic search in combinatorial optimization and SAT solvers. Qiu and Liang (2025) trained the neighbor-generating function in simulated annealing using RL across a variety of problem instances. Their method dynamically adapts the search policy based on experience, achieving notable improvements in solution quality and convergence compared to standard SA. Kurin et al. (2020) applied RL in combination with graph neural networks to guide branching decisions in SAT solvers. By learning to select variables that reduce search depth, their approach decreased the number of solver iterations and improved efficiency on benchmark instances.

These studies demonstrate that RL can learn effective policies to guide combinatorial search.

## 3 Problem Formulation

### 3.1 Problem

The **Maximum Satisfiability (MaxSAT)** problem can be defined as follows:

**Given:**

- A set of $n$ Boolean variables $X = \{x_1, x_2, \ldots, x_n\}$, where $x_i \in \{0, 1\}$.
- A Boolean formula $F(X)$ in Conjunctive Normal Form (CNF) consisting of $m$ clauses $C = \{c_1, \ldots, c_m\}$, each clause being a disjunction of literals over $X$.

**Objective:** Find an assignment $Y \in \{0, 1\}^n$ of the variables that maximizes the number of satisfied clauses:

$$\max_{Y \in \{0,1\}^n} \text{Sat}(Y), \quad \text{Sat}(Y) = \sum_{j=1}^{m} \mathbf{1}[c_j(Y) = 1],$$

where $\mathbf{1}[c_j(Y) = 1]$ is 1 if clause $c_j$ is satisfied under assignment $Y$, and 0 otherwise.

### 3.2 Evaluation Setup

We evaluate algorithms on multiple sets of randomly generated MaxSAT instances. Each set contains problem instances with:

- Number of variables $n$ ranging from 20 to 75.
- Number of clauses $m$ ranging from 71 to 325, with exactly 3 variables per clause.
- All instances are guaranteed to be satisfiable.
- Difficulty increases with the number of variables and clauses.

This setup allows us to assess algorithm performance across a spectrum of problem sizes and complexities, focusing on solution quality and robustness rather than variations across instance letters or labels.

## 4 Simulated Annealing Baseline

The simulated annealing (SA) algorithm we implement searches for high-quality variable assignments by exploring neighboring solutions in a stochastic manner. Starting from a random initial assignment of all $n$ Boolean variables, a *neighbor* is generated by selecting a single variable uniformly at random and flipping its value. The change in solution quality is measured as

$$\Delta = \text{Sat}(\text{neighbor}) - \text{Sat}(\text{current}),$$

where $\text{Sat}(\cdot)$ is the number of satisfied clauses.

Neighbor acceptance follows the classical *Boltzmann–Metropolis* criterion (Kirkpatrick et al., 1983). If $\Delta > 0$, the move is accepted unconditionally. If $\Delta < 0$, the move is accepted with probability

$$\exp(\Delta / T),$$

which decreases as the temperature $T$ cools, allowing the algorithm to escape local optima early and behave increasingly greedily as the temperature decreases.

The initial temperature $T_0$ is estimated using the Boltzmann acceptance rule (Kirkpatrick et al., 1983). We generate random solutions and their neighbors, compute the absolute changes in satisfied clauses, and take the mean. This average is scaled by the logarithm of the cooling rate to produce a temperature at which a typical negative $\Delta$ is accepted with probability approximately equal to the cooling rate:

$$T_0 = -\frac{\langle |\Delta| \rangle}{\ln r}.$$

During the search, the temperature is decreased multiplicatively by a cooling rate of $r = 0.99$, and a fixed number of neighbors, 40, are evaluated at each temperature. The algorithm terminates either when the temperature falls below 0.1 or when a fully satisfying assignment is found. Throughout the process, the best solution found is tracked and updated whenever a neighbor improves the number of satisfied clauses.

This SA implementation provides a simple, minimally tuned baseline, allowing a fair comparison to the DQN agent in terms of raw search performance rather than reliance on hand-crafted heuristics.

## 5 Deep Q-Network for Weighted Max-SAT

We use a Deep Q-Network (DQN) to select which variable to flip during the search. Directly providing the DQN with all variables, the clauses they appear in, and their current states would be computationally expensive and would not allow the model-free network to generalize across new instances where a variable does not correlate to its

predecessor from previous instance, so we represent the environment with a feature vector instead. For each variable $v$, the environment computes an *8-dimensional* feature vector that reflects how flipping $v$ would affect the current assignment:

- **Make count**: the number of currently unsatisfied clauses that would become satisfied if variable $v$ were flipped.
- **Break count**: the number of currently satisfied clauses that would become unsatisfied after flipping $v$.
- **Satisfaction delta**: a signed local change score reflecting the net impact on clause satisfaction.
- **Current variable value**: the current assignment of $v$ (either 0 or 1).
- **Complementary value**: the value $1 - \text{current}$.
- **Clause occurrence count**: the number of clauses in which $v$ appears.
- **Make–break difference**: the raw difference $\text{make} - \text{break}$.
- **Unsatisfied-clause fraction**: the global fraction of clauses that are currently unsatisfied.

Thus, the full state provided to the DQN is a matrix of shape

$$\text{num_vars} \times 8,$$

where each row corresponds to a possible flip action (i.e., flipping a variable).

### 5.1 Q-Network Architecture

For each variable $v$, its 8-dimensional feature vector is passed to a neural network that estimates the Q-value

$$Q_\theta(s, v).$$

The Q-network is a fully connected feedforward model consisting of:

- an input layer of 8 features,
- hidden layers with 256, 128, and 64 ReLU-activated units,
- a single linear output unit representing the predicted Q-value for flipping $v$.

A target network of identical structure is updated every 500 training steps to stabilize learning.

### 5.2 Action Selection and Training

At each step, the agent chooses an action using an $\epsilon$-greedy rule:

$$a = \begin{cases} \text{random variable}, & \text{with probability } \epsilon, \\ \arg\max_v Q_\theta(s, v), & \text{otherwise.} \end{cases}$$

The exploration rate begins at $\epsilon = 1.0$ and decays by a factor of $0.995$ until reaching a minimum of $0.05$. When training proceeds across multiple epochs, we reset $\epsilon$ to $1.0$ at the start of each epoch, giving the agent a second opportunity to explore while reusing its previously learned parameters.

Training uses a replay buffer of 50,000 transitions and minibatches of size 128. The Bellman target used for training is:

$$y = r + \gamma(1 - d)\, Q_{\hat{\theta}}(s'),$$

where $d$ is a terminal flag, $\gamma = 0.99$, and $Q_{\hat{\theta}}$ denotes the target network. We optimize using Adam with a learning rate of $10^{-4}$.

To ensure practical runtime, each instance is limited to **5000 flips**. Reward is computed from the change in the number of satisfied clauses, with small penalties for non-improving steps and a bonus for achieving full satisfaction.

### 5.3 Practical Limits and Learned Heuristics

Because the full Max-SAT search space is enormous relative to the number of training instances and epochs, the DQN does not learn a globally reliable policy. Instead, it acquires an *approximate heuristic* that captures local flip preferences from experience. We train for two epochs, resetting exploration between them to allow the network to refine its behavior while retaining learned weights. Despite computational limitations, the agent develops nontrivial flip patterns that differ from standard stochastic local search.

## 6 Experiments

We benchmark both Simulated Annealing (SA) and our DQN-based reinforcement learning (RL) approach on standard MaxSAT problem sets. Each set contains 100 instances generated randomly with roughly uniform difficulty.

SA serves as a baseline due to its simplicity and well-studied behavior. In our experiments, it typically caps at around 19,000 flips for instances it could not fully solve. This provides a consistent reference for evaluating the RL approach.

The DQN agent, in contrast, learns a policy to select promising variable flips, solving instances more efficiently. Training over repeated epochs on the same problem set allows the agent to progressively improve its decisionmaking and reduce the required flips.

### 6.1 Run Setup

We conducted six experiments covering benchmark families of increasing size and complexity:

- 20 variables, 71 clauses
- 20 variables, 91 clauses
- 36 variables, 157 clauses
- 50 variables, 218 clauses
- 75 variables, 325 clauses

Each DQN run completed 2 epochs over the available 100 instances. Exploration was decayed from $\epsilon = 1.0$ to $\epsilon_{\min} = 0.05$ with a factor of 0.995 per flip.

Although our main experiments use 100 instances per set, we verified that training on 200 instances produces comparable performance, indicating good generalization (see Appendix A.5 for details). The figures presented focus on the 100-instance setup. All code is publicly available on GitHub (Ohrablo, 2025) and can be run locally.

### 6.2 Results Overview

Figure 1 shows results for a representative problemset (36 variables, 157 clauses). Panels (a) and (b) display the distribution of satisfied clauses across all instances for SA and RL, respectively. The histogram shows most instances being fully satisfied with the RL agent (panel b) achieving slightly higher effectiveness.

Panels (c) and (d) display the total number of flips required per instance. The RL agent typically requires much fewer flips per instance showing sings of meaningful learning. The red vertical lines in panel (d) indicate the beginning of each training epoch.

### 6.3 Averages Across All Experiments

Table 1 summarizes averages across all six benchmark families. The table lists the method, problem set, number of instances, Average flips per instance, Average clauses satisfied, and the percentage of fully solved instances. DQN consistently requires fewer flips per solution while also being able to find the full solution for more instances of the harder problems, suggesting an ability of the network to find the correct strategy.

| Method | Set | N | Flips | Clauses | Solved |
|---|---|---|---|---|---|
| SA | 20-71 | 100 | 726.89 | 71.00 | 100% |
| DQN | 20-71 | 100 | 86.39 | 70.99 | 99% |
| SA | 20-91 | 100 | 14615.42 | 90.99 | 99% |
| DQN | 20-91 | 100 | 110.83 | 91.00 | 100% |
| SA | 36-157 | 100 | 18506.22 | 156.80 | 80% |
| DQN | 36-157 | 100 | 544.88 | 156.99 | 99% |
| SA | 50-218 | 100 | 18924.89 | 217.47 | 54% |
| DQN | 50-218 | 100 | 2963.13 | 217.95 | 95% |
| SA | 75-325 | 100 | 19167.98 | 324.14 | 37% |
| DQN | 75-325 | 100 | 3133.92 | 324.66 | 69% |

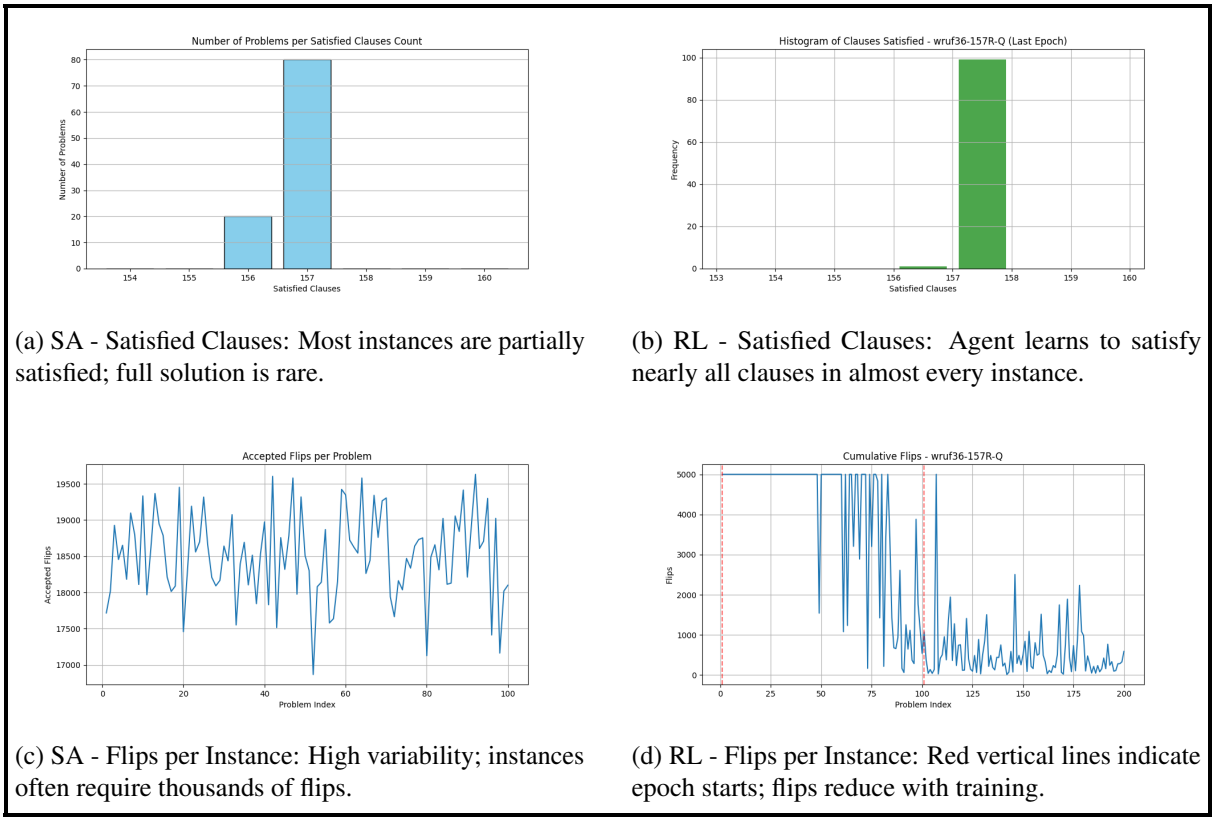
Table 1: Method, Problemset, N of problems, Average flips, average satisfied clauses, and full solution rate for SA and DQN across six MaxSAT problemsets. DQN numbers are from the second epoch.

## 7 Conclusion

In this work, we compared a classical Simulated Annealing (SA) approach with a Deep Q-Network (DQN)–based reinforcement learning method for solving MaxSAT instances. Our experiments demonstrate that, while SA provides a reasonable baseline, it is limited by its reliance on manually tuned parameters and can require a large number of flips to reach good solutions.

The DQN agent, in contrast, efficiently guides its search by learning which variable flips are most promising, achieving comparable or better solution quality with fewer flips. Even with only 2 epochs over the same set of instances, the RL approach was able to refine its policy and consistently reach high-quality assignments. This indicates that model-free RL methods can adapt to the structure of MaxSAT landscapes and serve as effective alternatives to classical local search techniques.

Future work could explore scaling to larger problem instances, integrating more sophisticated RL architectures, or combining learned policies with traditional heuristics to further improve efficiency and solution quality.




(a) SA - Satisfied Clauses: Most instances are partially satisfied; full solution is rare.

(b) RL - Satisfied Clauses: Agent learns to satisfy nearly all clauses in almost every instance.

(c) SA - Flips per Instance: High variability; instances often require thousands of flips.

(d) RL - Flips per Instance: Red vertical lines indicate epoch starts; flips reduce with training.

Figure 1: Comparison of SA and RL performance for 36 variables, 157 clauses.

## References


Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. 1983. Optimization by simulated annealing. *Science*, 220(4598):671–680.

Vitaly Kurin, Saad Godil, Shimon Whiteson, and Bryan Catanzaro. 2020. Improving sat solver heuristics with graph networks and reinforcement learning. In *OpenReview*.

Andrej Ohrablo. 2025. Max sat project. `https://github.com/ohraband/rlmaxssat`. Accessed: 2025-12-05.

Nathan Qiu and Daniel Liang. 2025. Reinforcement learning based simulated annealing. In *AAMAS 2025*, pages 1718–1726.


## A Full Experimental Results

This appendix contains the full plots for all remaining problemsets evaluated in our study:

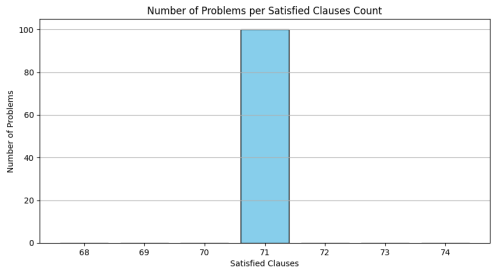
### A.1 20 Variables, 71 Clauses

### A.2 20 Variables, 91 Clauses

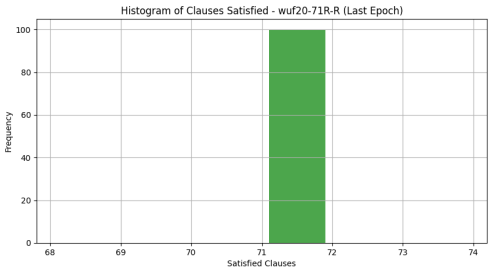
### A.3 50 Variables, 218 Clauses

### A.4 75 Variables, 325 Clauses

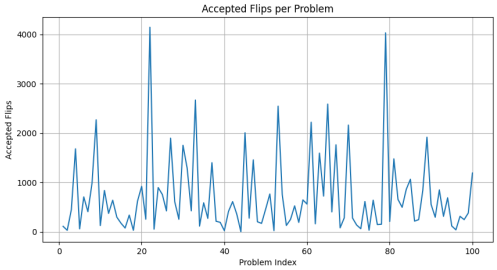
### A.5 75 Variables, 325 Clauses, 200 unique instances



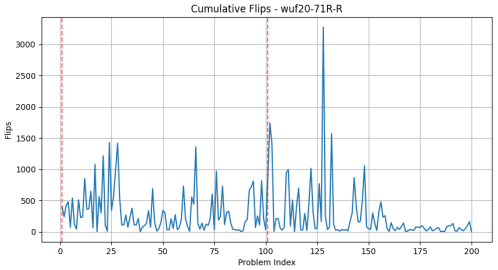

(a) SA - Satisfied Clauses




(b) RL - Satisfied Clauses

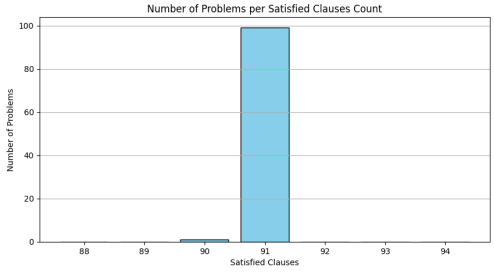



(c) SA - Flips

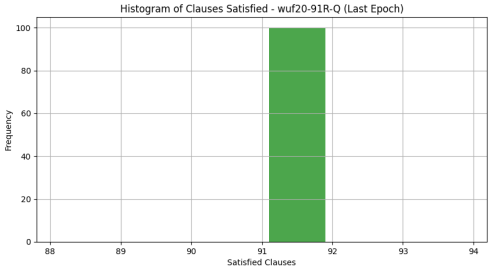



(d) RL - Flips

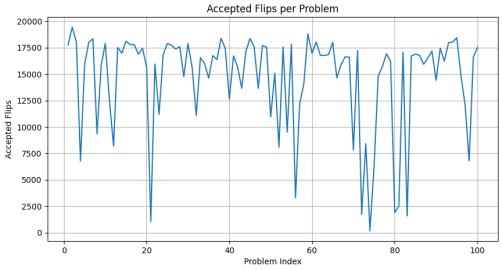
Figure 2: SA and RL performance for 20 variables, 71 clauses.



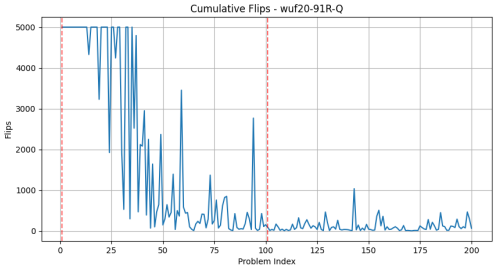

(a) SA - Satisfied Clauses




(b) RL - Satisfied Clauses

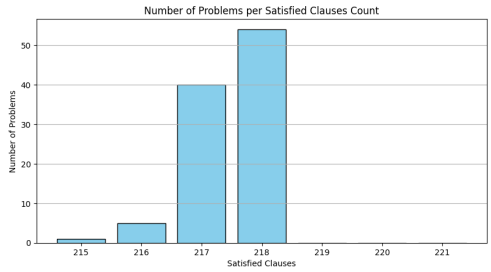



(c) SA - Flips

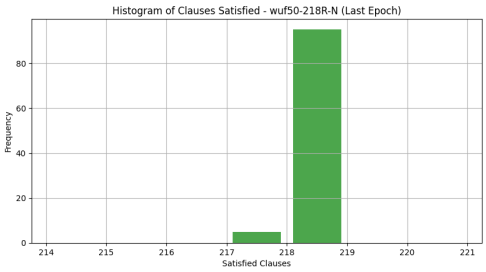



(d) RL - Flips

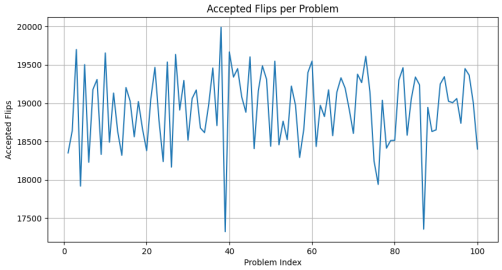
Figure 3: SA and RL performance for 20 variables, 91 clauses.



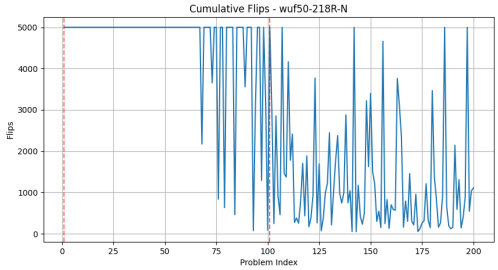

(a) SA - Satisfied Clauses




(b) RL - Satisfied Clauses

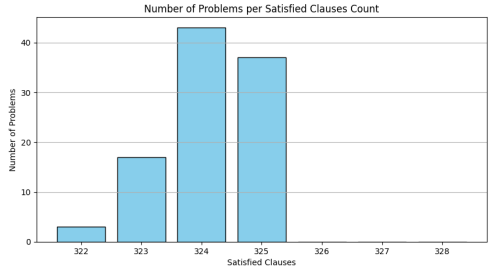



(c) SA - Flips

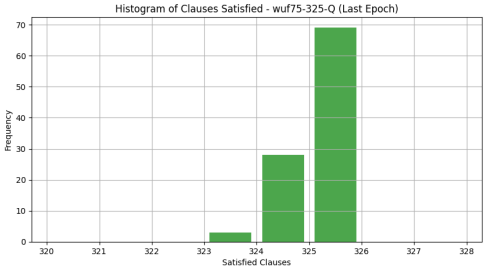



(d) RL - Flips

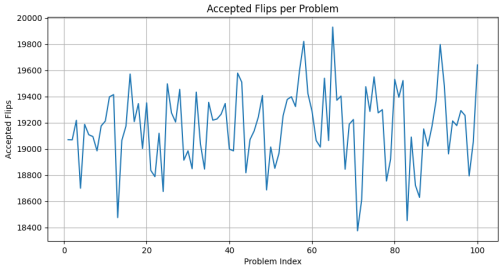
Figure 4: SA and RL performance for 50 variables, 218 clauses.



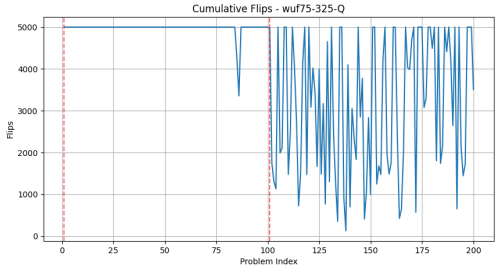

(a) SA - Satisfied Clauses




(b) RL - Satisfied Clauses

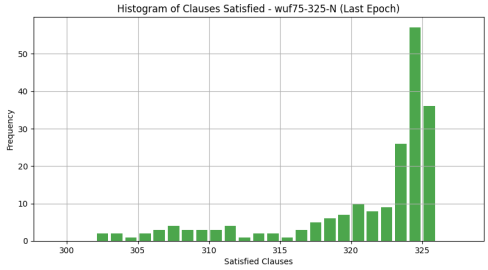



(c) SA - Flips

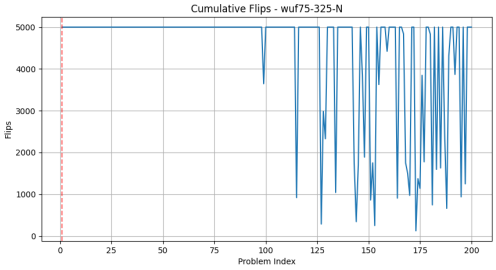



(d) RL - Flips

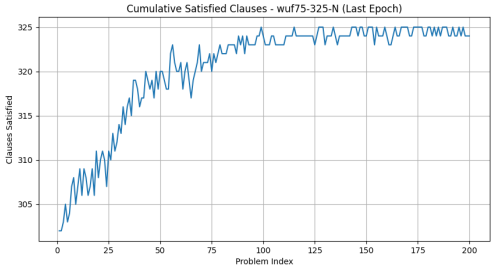
Figure 5: SA and RL performance for 75 variables, 325 clauses.




(a) RL - Satisfied Clauses




(b) RL - Flips




(c) RL - Satisfied Clauses Progression

Figure 6: SA and RL performance for 75 variables, 325 clauses. 200 unique instances.